Universal Teleoperation DM-EXton2 Robot Adaptation and Development Guide 2026210 (V1.1)

Universal Teleoperation DM-EXton2 Robot Adaptation and Development Guide 2026210 (V1.1)

Target Audience: Robot integration engineers, algorithm developers, and researchers

Confidentiality Level: Public

Last Updated: 2026-02-10

---

1. Overview (Overview)

1.1 System Introduction

This Software Development Kit (SDK) mainly consists of the following modules: Robot Feedback Joint Angle Conversion, DM-EXton2 Device Interface, and Pink IK Simulation Module. By integrating **Pink** inverse kinematics solver and **Pinocchio** dynamics library, this system can convert the end poses (Cartesian Poses) sent by DM-EXton2 into robot joint commands (Joint Commands) in real time, providing examples for customer adaptation. This case adapts two robots, namely Limx Oli EDU and Unitree G1-EDU.

1.2 Important Risk Statement

⚠[Important Warning] Regarding the /target_robot/joint_commands data calculated based on Pink simulation adaptation, we do not make any guarantees for the risks associated with using this data for real robot control. Users must fully understand the following risks and take appropriate protective measures.

Before using this system to control real robots, please be sure to carefully read and fully understand the following risk statement:

1.2.1 Limitations of the Pink Simulation Module

The Pink IK (Inverse Kinematics) simulation module included in this software development package is a simulation demonstration program developed based on the Pinocchio library and

the Pink framework. The main purpose of this module is to provide users with a simulation environment for quick verification and simulation testing, rather than for directly controlling real robots. The joint commands /target_robot/joint_commands calculated by the Pink module have the following limitations:

1. Differences between Simulation and Reality : The simulation environment uses idealized physical models and does not consider factors such as the dynamic characteristics, friction, clearance, and flexibility of real robots. Motion trajectories that are feasible in simulation may not be achievable or may result in unexpected behavior on real robots.

2. Missing safety limits: Although the Pink module includes basic configuration limits and speed limits, these limits are based on theoretical values in the URDF model. Real robots may have more stringent safety limit requirements, such as torque limits, acceleration limits, workspace limits, etc.

3. Unverified Algorithm: The quadratic programming-based IK solver used by Pink is mathematically rigorous, but it has not been fully tested and verified for your specific robot model. Different robot configurations, link parameters, and joint settings can all affect the effectiveness of IK solving.

4. Real-time performance not guaranteed: The Pink module runs in a ROS2 node and is affected by factors such as operating system scheduling, network latency, and computational load. In real robot control scenarios, jitter in the control cycle may lead to unstable behavior.

1.2.2 Recommended Usage

We strongly recommend that users use this system in the following manner:

Method 1: Use a custom IK algorithm (recommended)

If you have your own IK inverse kinematics algorithm or the robot manufacturer has relevant interfaces to accept end pose for control, after completing the input-output test of the exoskeleton interface, you can directly subscribe to the end target pose topics published by the exoskeleton (/target_robot/left_ee/target and /target_robot/right_ee/target), use them as inputs, calculate joint commands through your IK algorithm, and directly send them to your robot controller. This approach bypasses the Pink simulation module and is the most direct and reliable integration method.

Method 2: Deep customization based on Pink

If you wish to develop based on Pink, you need to deeply customize the Pink module, including: reconfiguring IK parameters according to your robot's URDF model, adding safety constraints specific to your robot, implementing more comprehensive error handling and exception recovery mechanisms, and conducting sufficient testing and verification. Even so, we still recommend that you conduct a professional safety assessment before deploying on a real robot.

Method 3: Only use the simulation function

As a simulation demo we provide to customers, the main value of the Pink module lies in: helping users quickly understand the working principle of the system, providing a runnable reference implementation, and supporting functional demonstration and development testing without a real robot. In the simulation environment, you can safely verify teleoperation logic, debug communication interfaces, and develop upper-level applications.

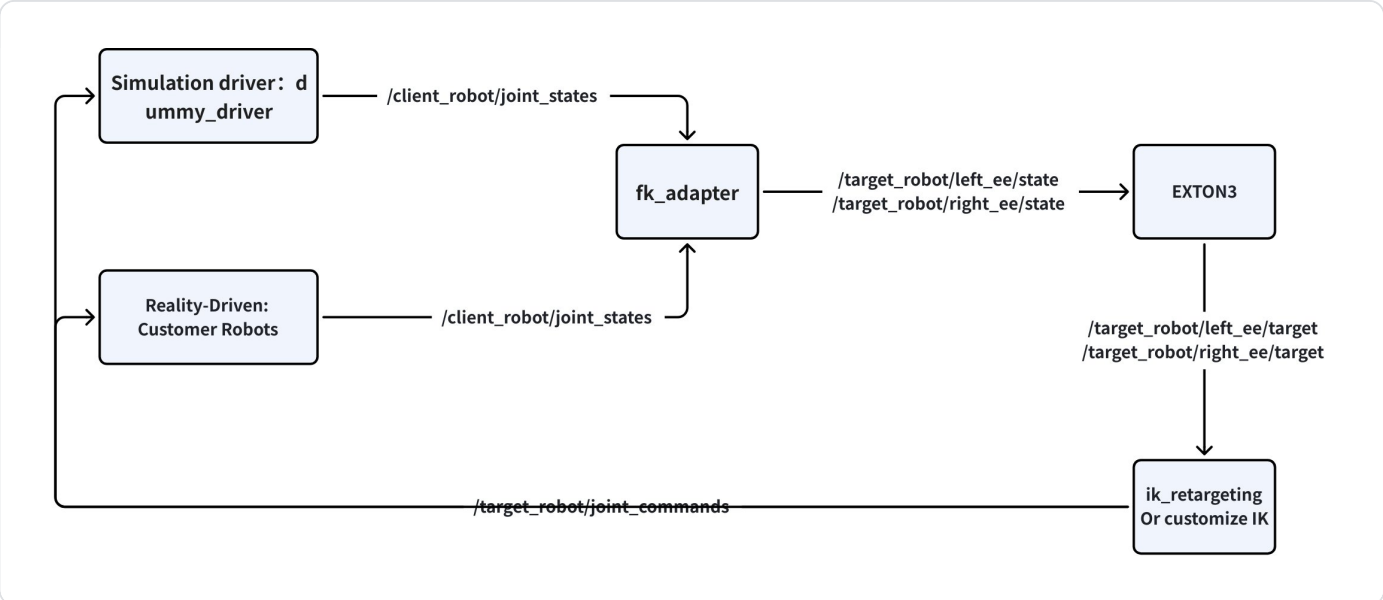
---

2. System Architecture (System Architecture)

The system consists of three core nodes, communicating via ROS 2 topics and TF transformation trees.

2.1 Data Flow Diagram

1. **Feedback Link (Robot -> Exoskeleton)**:
   - `DummyRobotDriver` (or real driver) publishes joint states `/client_robot/joint_states`.
   - `robot_state_publisher` converts joint states into a TF tree.
   - `JointToPoseAdapter` listens to TF, calculates the end pose, and publishes it to `/target_robot/{side}_ee/state`.



2. **Control Link (Exoskeleton -> Robot)**:
   - EXton2 publishes the target pose to `/target_robot/{side}_ee/target`.
   - `/ik_retargeting` node or custom ik receiving target, combined with the current state, calculates the joint position.

- The solution results are published to `/target_robot/joint_commands` .

2.2 Node List

Node Name	corresponding file	Function Description
ik_retargeting	ik_node.py	Core IK Solver, supporting relative control and anthropomorphic posture.
fk_adapter	joint_to_pose_adapter.py	Forward kinematics adapter, which converts jo end-effector pose.
dummy_driver	dummy_robot_driver.py	Virtual driver, used for simulation testing in a h environment.

2.3 Explanation of Core Topics

The system defines the following core ROS2 topics. Understanding the purpose of these topics is the foundation for system adaptation:

Topic Name	Message Type	Frequency/Hz	Description
/client_robot/joint_states	JointState	——	Robot's current joint state (output)
/target_robot/joint_commands	JointState	250	Target Joint Command (Input)
/target_robot/left_ee/target	PoseStamped	500	Left end target pose (output)
/target_robot/right_ee/target	PoseStamped	500	Right end target pose (output)
/target_robot/left_ee/state	PoseStamped	——	Left End Current State (Input)
/target_robot/right_ee/state	PoseStamped	——	Current state (input) of the right end
/clutch/left	Bool	1000	Left Clutch Enable Signal
/clutch/right	Bool	1000	Right Clutch Enable Signal

In the following chapters, we will introduce in detail the working principles, interface specifications, and adaptation methods of each component. Readers are requested to read carefully to ensure a full understanding of each link.

3. Quick Start

3.1 Environmental Dependencies

- **Operating System**: Ubuntu 22.04 (Recommended)
- **ROS Version**: ROS 2 Humble/Iron
- **Core library**: `pinocchio`, `pink`, `numpy`, `tf2_ros`

3.2 Install system-level dependencies

First, update the software sources and install Pinocchio and related ROS plugin dependencies:

Install system-level dependencies

```
sudo apt update
sudo apt install ros-humble-pinocchio
sudo apt install ros-humble-eigen3-cmake-module
```

3.3 Configure a Python Virtual Environment

To avoid dependency conflicts, it is recommended to install pink and its required dependencies in a virtual environment.

3.3.1 Install the venv module (if not installed in the system)

Install the venv module

```
sudo apt install python3-venv
```

3.3.2 Create a Virtual Environment

Create a virtual environment named `pink` (or other custom name) in the project root directory:

Creating a virtual environment

```
cd exton_ros2_retargeting_demo
python3 -m venv pink
```

Pay attention to check whether the created virtual environment is Python 3.10.

3.3.3 Activate the Virtual Environment

IK inverse kinematics requires pink dependency, and the virtual environment needs to be activated before each IK inverse kinematics operation

Activate virtual environment

```
cd exton_ros2_retargeting_demo
source pink/bin/activate
```

> **Note**: After successful activation, the `(pink)` identifier will appear before the command prompt.

3.4 Install project dependencies

In the activated virtual environment, use `pip` to install Python dependency packages:

Install project dependencies

```
cd exton_ros2_retargeting_demo
source pink/bin/activate
# requirements.txt在发货U盘的exton_ros2_retargeting_demo文件中
pip install -r requirements.txt
```

🚀 **Acceleration Installation Prompt:**
If the download speed is slow, you can use the Tsinghua University mirroring source for acceleration:

Bash

```
pip install -r requirements.txt -i https://pypi.tuna.tsinghua.edu.cn/simple
```

3.5 Compile

Ensure the code is located under the workspace `src` directory, then execute:

Bash

```
colcon build
source install/setup.bash
```

3.6 Run the Simulation Demo

Launch the base environment containing the FK adapter, virtual driver, and RViz:

```
# Limx Oli EDU
ros2 launch robot_fk_adapter robot_fk_adapter.launch.py use_sim:=true
# Unitree G1-EDU
ros2 launch robot_fk_adapter robot_g1_fk_adapter.launch.py use_sim:=true
```

Note: This Launch file will load the URDF of `HU_D04_description`.

Start the IK Solver Core:

Bash

```
# Limx Oli EDU
ros2 launch robot_ik_retargeting ik_retargeting.launch.py
# Unitree G1-EDU
ros2 launch robot_ik_retargeting g1_ik_retargeting.launch.py
```

---

4. Robot Adaptation Guide (Adaptation Guide)

This is the core part of this manual. If you need to adapt a new robot (such as your custom robotic arm or humanoid robot), please follow the steps below strictly.

4.1 Step 1: Prepare URDF

The system fully relies on URDF (Unified Robot Description Format) for kinematic calculations.

1. Ensure that your robot description package (Description Package) and this SDK are placed in the same workspace and have been compiled.
2. Confirm that the URDF contains accurate kinematic chains and joint limits.
3. **Key Check**: Confirm the root node name of the URDF (usually `base_link`) and the node name you wish to use as the reference for relative control (such as `waist_pitch_link`).

4.2 Step 2: Configure the FK Adapter

Modify the parameters in `robot_fk_adapter.launch.py` to match your robot's TF tree.

- **base_frame**: Set to the base coordinate system of the robot (e.g., `base_link`). FK will calculate the end coordinates based on this. Note to correctly input the reference coordinate system for the ik inverse solution. If your custom ik algorithm is based on the inverse solution of the base coordinate system, please input the base coordinate system of the robot (e.g., `base_link`). If your custom ik algorithm is based on the inverse solution of the torso's waist coordinate system, please input the waist coordinate system of the robot (e.g., `waist_pitch_link`).

- **Note**: If you need to use the Pink inverse kinematics algorithm, you can only use the robot's base coordinate system (such as `base_link`)
- **left_ee_frame / right_ee_frame**: Set to the Link name of the robot's left and right end-effectors (e.g., `left_hand_link`).

Python

```python
# Example: Modify robot_fk_adapter.launch.py
Node(
    package='robot_fk_adapter',
    executable='joint_to_pose_adapter',
    parameters=[{
        'base_frame': 'base_link',             # Edit here
        'left_ee_frame': 'link_l_hand',        # Edit here
        'right_ee_frame': 'link_r_hand'        # Edit here
    }]
)
```

- **If driving a real robot, you need to modify `use_sim` in `robot_fk_adapter.launch.py` to `false`, otherwise the simulation driver node `dummy_driver` will conflict with the real robot**

robot_fk_adapter.launch.py

```
return LaunchDescription([
        DeclareLaunchArgument('use_sim', default_value='false',
description='Set to false for real robot'),modify here default_value='false'
        DeclareLaunchArgument('use_rviz', default_value='true'),
```

4.3 Step 3: Configure the IK Solver Core (skip if using a custom IK algorithm)

This is the most critical step, which determines the solution effect and the naturalness of the pose. Please edit `ik_retargeting.launch.py` file.

4.3.1 Basic Link Configuration

Set URDF path and key Link name:

Python

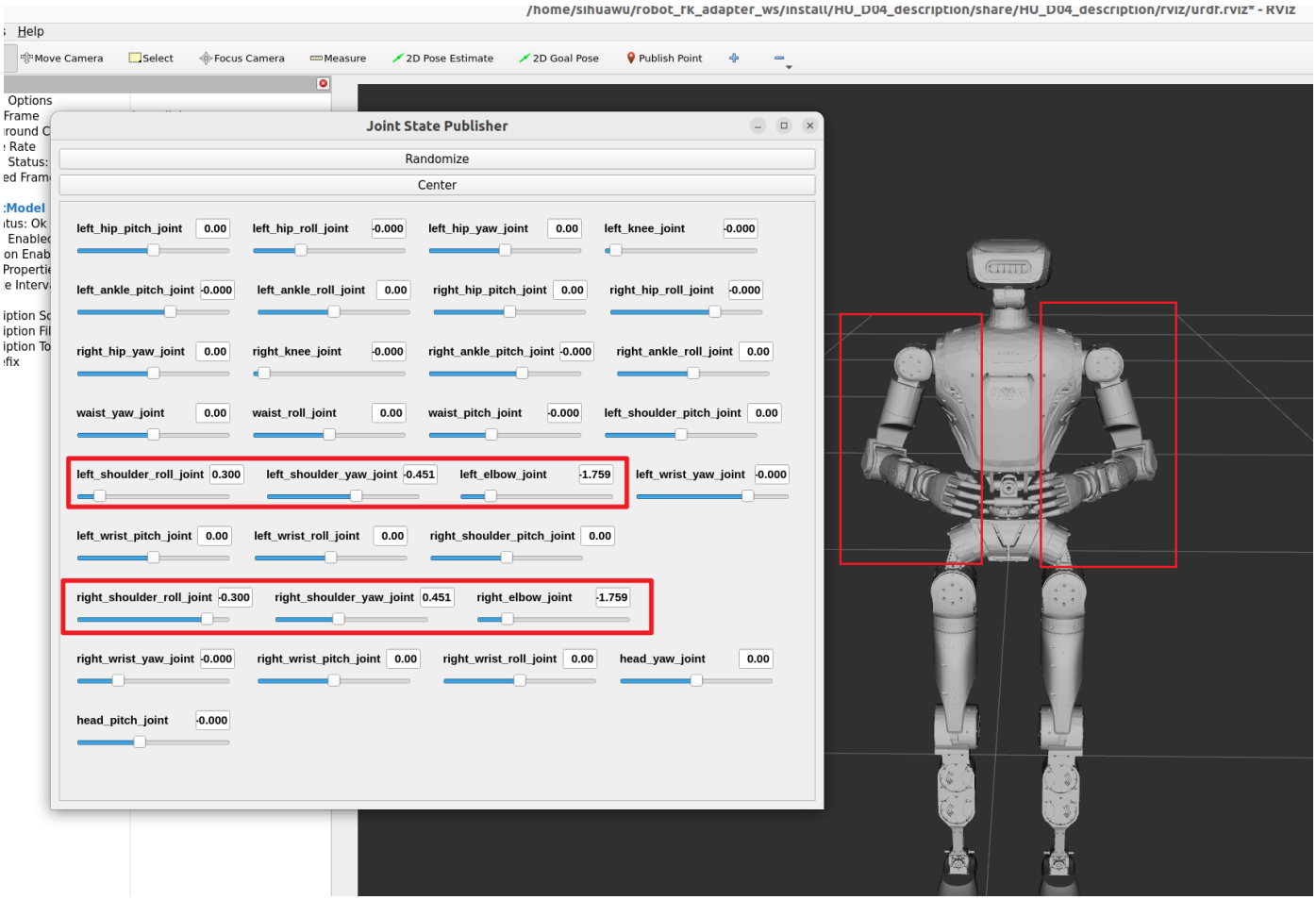
```
# 1. Set the description package name and URDF relative path
pkg_name_arg = DeclareLaunchArgument('description_package',
default_value='my_robot_description')
urdf_file_arg = DeclareLaunchArgument('urdf_relative_path',
default_value='urdf/my_robot.urdf')
```

```
4
5   # 2. Set the control target Link (must be consistent with URDF)
6   default_target_links = ["link_r_hand", "link_l_hand"]
```

4.3.2 Human-like Pose Tuning (Human-like Config)

To make the robot's posture more anthropomorphic, it is necessary to configure `preferred_joints` and `preferred_angles`.

- **Principle**: The system will apply a weak force in the null-space to pull the joint towards the set desired angle.
- **Debugging Method** :
  a. Find the joint names of the robot's shoulder (Shoulder) and elbow (Elbow).
  b. Open the robot Rviz simulation, drag the relevant joints, and move the robot to an anthropomorphic elbow posture.



c. Determine the desired angle (in radians). For example, the natural bend of the elbow is typically 90 degrees (± 1.57 rad).
d. **Pay attention to the sign** : The motor installation directions of different robots vary. If the elbow bends backward, please reverse the sign.

Configuration Example:

```python
# Modify the launch file
preferred_joints_default = [
    "left_shoulder_roll", "left_shoulder_yaw", "left_elbow",  # 左臂关节
    "right_shoulder_roll", "right_shoulder_yaw", "right_elbow" # 右臂关节
]

# Corresponding Expectation Angle
preferred_angles_default = [
    0.3,   # The robot's left shoulder spreads outwards
    0.0,   # The robot's left shoulder rotates back to center
    -1.5,  # The robot's left elbow is bent at 90 degrees.
    -0.3,  # The robot's right shoulder spreads out symmetrically.
    0.0,
    -1.5   # The robot's right elbow bent
]
```

Warning: The list length of `preferred_joints` and `preferred_angles` must be strictly consistent, otherwise the node will report an error.

---

5. Detailed Explanation of Pink's ik_retargeting Function (Detailed Features)

When using the custom ik algorithm for control, it can be referred to that when the enable button (Clutch) of the remote control handle is not pressed, the end target pose of the robot will not be sent during remote operation, and the target pose will only be sent after it is pressed:

5.1 Soft Start Logic (Soft Start)

When the enable key (Clutch) of the remote control handle is pressed, the system will not immediately jump to the target position, but instead trigger the soft start process.

- **Trigger Condition**: `/clutch/{side}` topic receives the rising edge of the `True` signal.
- **Execution Process**:
  a. Record the current physical location as an anchor point (Anchor).
  b. Reset the speed cache to prevent differential shock.
  c. Within `0.5` seconds, smoothly transition the target from the anchor point to the EXton2 instruction using an interpolation algorithm.

d. During this period, the Damping weight is temporarily increased to `50.0` to suppress oscillations.

e. After the transition ends, resume normal tracking mode, and the damping returns to `20.0`.

5.2 Finite-State Machine Behavior

The system maintains a simple finite-state machine in `ik_node`:

Status	Trigger Condition	Behavior Description	Output Target
Locked	Clutch = False	The robot maintains its current posture and does not respond to the EXton2 action.	Current Physical Pose (T_current_phys)
Soft Start	Clutch Rising Edge	Smooth transition within 0.5 seconds, large damping.	Interpolate(Anchor, Target, alpha)
Tracking	Clutch = True (t > 0.5s)	Real-time response to EXton2 target, including anthropomorphic optimization.	EXton2 Goal (T_topic)

5.3 Internal Algorithm Parameter Tuning

Some advanced parameters are located inside the code and need to be modified directly `ik_node.py` file to meet different tactile requirements.

A. Filtering Parameters (One Euro Filter)

Location: `init` function `self.filters` definition

- `mincutoff` **(minimum cutoff frequency)**: Controls **stability at rest**.
  - **Increase** : Faster response, but may shake when the hand is still.
  - **Turn down**: Extremely stable, but with a "drag feeling" delay.
- `beta` **(speed coefficient)**: Controls **the response during rapid movement** .
  - **Increase** : The dynamic response becomes faster, but overshoot may occur.
  - **Turn down**: Dynamic response slows down, like moving in water.

B. IK Task Weights

Location: `init` function `self.tasks` definition

- `ee` **(end task)**:
  - `position_cost=300.0` : Priority of position tracking.
  - `orientation_cost=30.0` : Priority of pose (rotation) tracking. **Decreasing this value makes the wrist more flexible and reduces wrist "deadlock" phenomenon.**
- `posture` **(posture task)**:
  - `cost=1.0` : **Key Parameter**.
  - **Increase** : The robot will forcibly maintain the "natural posture", which may cause the hand to be unable to reach the target.
  - **Reduce**: The robot arm can reach farther, but the elbow posture optimization may not be adequate.
- `damping` **(damping task)**:
  - `cost=20.0` : Prevents singularity runaway. Generally not recommended to modify.

5.4 TF Visualization

For ease of debugging, the IK node broadcasts in real-time TF transforms named `ik_target_left` and `ik_target_right`.

- **Parent Frame**: `root_link_name` (如 `base_link`)。
- **Usage**: In RViz, you can directly see the virtual target point that the IK solver is currently "chasing". If the robot doesn't move but the TF does, it indicates that the IK solver parameters are too rigid or the dynamic constraints are in effect; if the TF is erratic, it indicates that the EXton2 data is abnormal.

---

6. Interface Definition (API Reference)

6.1 Subscribed Topics (Subscribed Topics)

- `/client_robot/joint_states` (`sensor_msgs/JointState`): The current real joint angles of the robot.
- `/target_robot/left_ee/target` (`geometry_msgs/PoseStamped`): Left hand target pose sent by EXton2.
- `/target_robot/right_ee/target` (`geometry_msgs/PoseStamped`): Right hand target pose sent by EXton2.
- `/clutch/left` (`std_msgs/Bool`): Left hand enable signal.
- `/clutch/right` (`std_msgs/Bool`): Right hand enable signal.

6.2 Published Topics

- `/target_robot/joint_commands` (`sensor_msgs/JointState`): The calculated target joint angles, directly sent to the underlying driver.
- `/target_robot/{side}_ee/state` (`geometry_msgs/PoseStamped`): The current end-effector pose calculated by the FK adapter.
- `/tf` : Broadcast `ik_target_{side}` for visualization.

---

7. Troubleshooting

7.1 Startup error: `Mismatch: preferred_joints vs preferred_angles`

Reason: The number of anthropomorphic joint names configured in the Launch file does not match the number of angles.

Solution : Check `ik_retargeting.launch.py` to ensure that the lengths of the two lists are exactly equal.

7.2 Startup error: `Parameter type error` (BYTE_ARRAY)

Reason: Type inference bug when passing an empty list as a ROS 2 parameter.

Solution : This issue has been fixed in the code via `ParameterDescriptor` . Please ensure that an empty list `[]` is not passed in the Launch file. If anthropomorphic configuration is not required, do not declare these two parameters in the launch file and let them remain the default `None` .

7.3 The robot arm has an abnormal posture (e.g., reverse joint)

Reason: `preferred_angles` The angle direction set is opposite to the actual positive direction of the motor.

Solution : Observe the joint axis in RViz and modify the angle sign in `ik_retargeting.launch.py` . For example, change `-1.5` to `1.5` .

7.4 The robot shakes violently at the moment of enabling

Reason: The EXton2 coordinate system is not aligned with the robot coordinate system, or the initial error is too large, exceeding the processing capacity of soft start.

Solution :

1. Check if `joint_to_pose_adapter` has correctly published the State.
2. Ensure that the EXton2 end uses this State as the calculation basis for the initial offset.
3. Increase the `soft_start_duration` (default 0.5s) in the code.

---

8. Appendix: Core Logic Code Snippet

8.1 Anthropomorphic Pose Construction

The system will dynamically construct `q_natural` vector based on the incoming parameters:

```python
# Iterate through the parameter list, overriding the default zero-position attitude.
if self.models[side].existJointName(j_name):
    jid = self.models[side].getJointId(j_name)
    idx_q = self.models[side].joints[jid].idx_q
    self.q_natural[side][idx_q] = angle
```

8.2 IK Task Stack Weight Allocation

```python
# End-point tracking task (high weight): Strictly execute position commands
"ee": FrameTask(self.ee_frames[side], position_cost=300.0, orientation_cost=30.0),

# Pose biasing task (low weight): Optimize pose without affecting the terminal effect.
"posture": PostureTask(cost=1.0),
```